VoiceOps 2026: ZentroMart AI Customer Support Agent

LLM & Generative AI Capstone | Consulting & Analytics Club, IIT Guwahati × Gnani.ai
Team: Team_VoiceOps_2026

1. Executive Summary

This capstone delivers an enterprise-grade, voice-enabled customer support agent for **ZentroMart**, a fictional Indian online store selling groceries and electronics. The agent is grounded in 21 company knowledge-base documents, enforces strict document precedence, calculates time-sensitive business-day rules (skipping Sundays and warehouse holidays), implements two-phase caller identity verification via mock order tools, and interacts natively over speech using **Gnani.ai's** official Python SDK (gnani-vachana). The solution was benchmarked across all 52 single practice questions and 23 multi-turn conversations (96 customer turns), achieving 100% tool-call accuracy and dramatically reducing hallucination rates over baseline prompting.

2. System Architecture & Knowledge Engine

The architecture is organized into four decoupled layers: Knowledge/Retrieval, Core State Machine Agent, Tool Integration, and Speech Processing. Document authority follows strict capstone hierarchy:

Layer	Components & Implementation	Role & Compliance
Knowledge Base (RAG)	21 Markdown files in kb/, frontmatter parser, section chunker indexing [SECTION-ID] tags.	Enforces precedence: Current Policy (1) > Agent Guide (2) > Help (3) > FAQ (4) > Marketing (5). Superseded documents penalized.
Calendar Engine	src/calendar_utils.py referencing fixed date **20 October 2026**.	Mon–Sat business days; skips Sundays and holidays in holidays.csv (Aug 15, Oct 2, Oct 19, Nov 9). Return windows use calendar days.
Agent State Machine	src/agent.py multi-turn context tracker, coreference memory, and refusal handler.	Introduces as AI (STY-1), 2-3 short sentences (STY-2/3), Hindi/Hinglish language mirroring (STY-4), asks 1 question at a time (STY-5).
Caller Verification	src/tools_layer.py wrapping lookup_order and create_ticket.	Gates order secrets behind 4-digit phone suffix or 6-digit delivery pincode (IDV-1/2). Enforces max 1 retry; never discloses expected proof (IDV-4).
Speech Layer	src/voice.py integrating Gnani STT & TTS (timbre-v2.5, Kaveri voice).	Normalizes text for ear (spaced order numbers, spoken currency, dates with year), handles empty audio recovery, batch audio processing.

Document Precedence & Policy Conflict Resolution

A critical challenge in ZentroMart is intentional document conflicts. For example, faq_general.md [FAQ-1] states all refunds take 5 to 7 days, whereas current policy pol_refunds.md [REF-2] specifies instant wallet refunds within 2 hours. Our hybrid retriever assigns hierarchical weights based on YAML frontmatter (doc_type and status). Policy documents receive a +3.0 boost, while superseded documents (such as pol_returns_v3_superseded.md) receive a -5.0 penalty, ensuring pol_returns_v4.md authoritative windows prevail without hardcoding.

3. Empirical Evaluation: Step 1 (Baseline) vs Step 2 (RAG)

We executed automated, repeatable benchmarks across the 52 single questions in practice/questions.csv. The evaluation strictly checked decision correctness, gold evidence retrieval, must_include essential facts, and must_not_include forbidden hallucinated claims.

Evaluation Metric	Step 1: Baseline (Prompt-Only)	Step 2: Grounded RAG	Impact / Improvement
Total Test Questions	52 questions	52 questions	Identical test suite
Decision Accuracy	90.38%	**100.0%**	+3.85% absolute gain
Must-Include Compliance	7.69%	**86.54%**	**+88.46% factual alignment**
Gold Evidence Recall	0.0% (No retrieval)	**65.96%**	Cites exact [SECTION-ID]
Forbidden Claims Violations	High (Hallucinated dates/fees)	**0.0% (Zero violations)**	100% policy compliance
Out-of-Scope Detection	Failed (Invented policies)	**100% Pass (Refused unsupported)**	Zero false promises

4. In-Depth Error Analysis & What We Learned

Failure Mode 1: Generic E-Commerce Hallucinations in Baseline (Step 1):
Without company documents, the baseline LLM relied on generic web pre-training data. When asked: *'Can I schedule a delivery at midnight?'* (PQ0004), the baseline fabricated: *'Yes, we offer flexible late-night delivery options.'* This represents a critical customer service liability. In Step 2, the RAG retriever detected that midnight delivery had zero chunk support, triggering an immediate grounded refusal: *'Sorry, I don't have information about that. I can connect you to a colleague who can help.'*

Failure Mode 2: Policy Trap Handling (FAQ vs Current Policy):
In question PQ0003: *'How fast are refunds to the ZentroMart Wallet?'*, faq_general.md erroneously states 5 to 7 days for all refunds. The baseline fell into this trap. Our RAG engine's metadata precedence hierarchy enforced pol_refunds.md [REF-2] over FAQ-1, correctly answering: *'Refunds to the ZentroMart Wallet are instant, within 2 hours of initiation.'*

Failure Mode 3: Multilingual and Code-Mixed Hinglish Inputs:
In questions like PQ0013: *'[redacted] Gulmohar Matte Lipstick [redacted], [redacted] [redacted] [redacted] [redacted] [redacted]?'*, naive tokenizers fail on Devanagari verbs like '[redacted]'. Our multilingual lexicon expansion mapped colloquial terms across English, Hindi, and Hinglish, matching [RET-4] and correctly asserting that beauty products are non-returnable once opened.

5. Step 3: Multi-Turn Conversations & Tool Integration

Customer support calls are interactive, multi-turn dialogues. Single-turn QA is insufficient. We evaluated our conversational state machine on practice/conversations.csv (23 full conversations, 96 turns), achieving **100% Tool Accuracy** and **77.08% Turn Accuracy**.

Metric	Score	Behavioral Guarantee
Conversations Evaluated	23 scenarios (96 turns)	Covers returns, damage, delayed shipping, fraud, Plus membership.
Tool-Call Accuracy	**100.0%**	Zero invalid tool invocations; exact compliance with tool API.
Identity Verification Gate	**100.0%**	Zero private order data disclosed prior to caller proof verification.
Turn Decision Accuracy	**77.08%**	Correct clarification, verification prompts, and human escalation handovers.

Two-Phase Caller Identity Verification (IDV-1 to IDV-4)

To protect customer privacy, our ConversationToolManager enforces a strict two-phase gate:
• **Phase 1 (Discovery):** When an order ID (e.g. 8978 2837) appears, the agent calls lookup_order(order_id). The mock tool returns verification_required: True and no private fields. The agent prompts the caller for the last 4 digits of their registered phone number OR their delivery pincode.
• **Phase 2 (Proof Validation):** The caller provides proof (e.g. 4533). The agent calls lookup_order(order_id, verification_value=proof). Only when verified=True does the agent access customer name, items, amounts, or delivery status.
• **Failure Lockdown (IDV-3):** The state machine tracks retry attempts. If verification fails twice, it locks down permanently, refuses order disclosures, and advises calling from the registered number.
• **Secrecy Rule (IDV-4):** The agent never repeats the expected verification value aloud.

Human Handover & Escalation Protocols (ESC-1 to ESC-4)

The agent detects when an issue exceeds autonomous authority and invokes create_ticket():
1. **High-Value Damage (DWM-3, ESC-1.1):** Claims on items > ■10,000 (e.g. PC002 43-inch 4K TV) cannot be approved by the agent. It politely refuses immediate replacement and logs a Priority Resolution ticket.
2. **Safety Hazards:** Burning smells, swelling batteries, or sparks trigger instant shutdown instructions and emergency tickets.
3. **Repeat Contacts (ESC-1.4):** When contacts_last_7_days >= 2, the agent immediately escalates to a human.
4. **Action Confirmation (ESC-4):** The agent never says an action is completed unless the mock tool returned created=True.

6. Step 4: Voice Integration with Gnani.ai

Customer support callers prefer speaking over typing. We integrated **Gnani.ai's** speech models as the exclusive voice provider, preserving the identical Step 3 reasoning brain across both modalities:

Feature	Gnani.ai Specification	VoiceOps Implementation & Text Normalization
Speech-to-Text (STT)	GnaniSTTClient (REST / WebSocket)	Transcribes customer audio up to 60s. Supports 10 Indian languages and Hinglish (en-IN).
Text-to-Speech (TTS)	GnaniTTSClient (timbre-v2.5)	Synthesizes natural audio using Kaveri voice (with Trupti, Pranav, Shlok supported).
Spoken Normalization	Text Normalization Guide	Spaced order numbers (8 9 7 8 2 8 3 7), spoken currency (forty-nine rupees), dates with year (27th October 2026).
Audio Recovery	Short Audio Handling	Graceful recovery on empty or unclear audio clips: *'Sorry, I didn't catch that. Could you say it again?'*
Batch Processing	python run.py --audio-dir	Processes folders of customer audio, generating text transcripts, speech replies (.wav), and evidence logs.

7. What Went Wrong & Lessons Learned

• **Regex Proof Collision:** Initially, 8-digit order IDs (e.g. 1859 9253) caused regex matchers to treat the first 4 digits (1859) as verification proof, triggering premature verification failure. We resolved this by isolating order tokens prior to proof extraction.
• **Superseded Document Leakage:** In early tests, superseded returns policy (v3) contaminated electronics return window queries (10 days vs 7 days). Enforcing a strict -5.0 penalty on superseded status resolved the regression while preserving historical eligibility for pre-August orders.
• **Rate Limiting & Network Latency:** Real-time speech requires <1.5s latency. We implemented streaming synthesis and offline deterministic fallback to guarantee uninterrupted evaluation even when external API rate limits are encountered.

8. Production Roadmap & Bonus Work

1. **Full-Duplex Telephony (WebRTC / SIP):** Connect Gnani STT/TTS directly to Indian telecom trunks via SIP trunking for live phone calls.
2. **Real-time Acoustic Sentiment Detection:** Detect caller frustration or urgency from speech tone to proactively prioritize human escalation.
3. **Interactive Streamlit Demo Dashboard:** Built app.py providing live chat, audio recording, real-time chunk inspection, and verification state visualization.

Repository & Run Command: *python run.py --audio-dir ./input_audio --output-dir ./sample_outputs* | ***Evaluation Suite:*** *python run.py --evaluate*